

Soundarray Tech Report

Concept-first architecture for spatial audio + classification

Simon Knight

2026-03-11 • Version 0.2

Last updated: March 11, 2026

Abstract. soundarray explores spatial audio processing and sound classification with an emphasis on edge capture (Raspberry Pi + microphone arrays) and optional remote compute. The design goal is inspectable, reproducible inference: signals are windowed explicitly, intermediate representations are named, and each stage has testable invariants.

Contents

1	Problem Statement	1
2	System Model (Concept-First)	1
2.1	End-to-End Dataflow: Predicates and Signal Pipelines	1
2.2	Consistency Boundaries: Snapshot-Then-Apply	1
3	Core Invariants (What Must Always Hold)	2
4	Algorithmic Spine (Advanced Ideas)	2
4.1	TDoA via GCC-PHAT (Why It Works)	2
4.2	Beamforming as Rewrite Semantics	2
4.3	Classification as a Gated Predicate	2
5	Deployment Shapes (Edge vs Remote)	3
6	Observability (What To Measure)	3
7	Open Questions / Next Steps	3

1 Problem Statement

We want a system that can (a) capture multi-channel audio from an array, (b) infer spatial properties (e.g., direction-of-arrival), (c) denoise/enhance, and (d) classify sources (vehicles, wildlife, etc.) under real-world constraints.

2 System Model (Concept-First)

The system is best understood as a set of signal transformations plus a small number of serialization points (places where we intentionally collapse concurrency into a stable, deterministic order).

Insight:
Prefer diagrams of predicates, invariants, and consistency boundaries. Avoid UML class diagrams and linear flowcharts: they hide the actual correctness contract.

2.1 End-to-End Dataflow: Predicates and Signal Pipelines

Use this as the mental model: a *predicate* (“there is a source of type X at bearing θ ”) is derived from multiple windowed signals and gates.

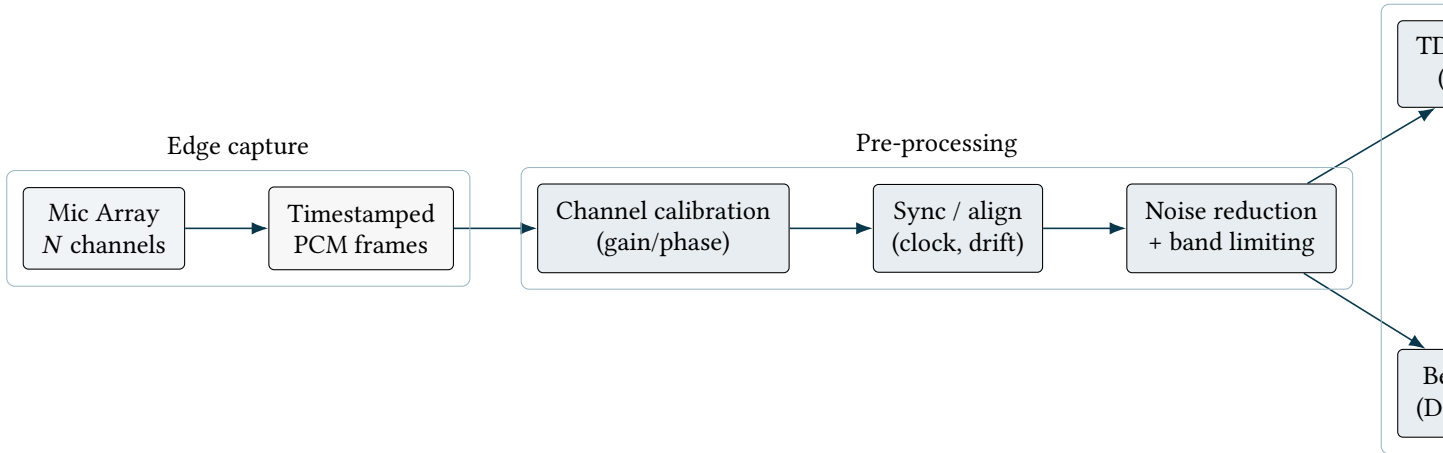


Figure 1. Predicate-first pipeline. The output is a provenance-bearing predicate over time (events), not “a label”.

2.2 Consistency Boundaries: Snapshot-Then-Apply

Spatial inference is sensitive to window boundaries. We treat each analysis tick as: (1) snapshot a well-defined window $W = [t_0, t_1]$, then (2) apply derived updates atomically.

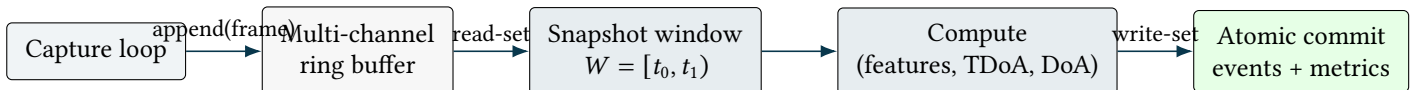


Figure 2. A single serialization point: snapshot a coherent window, then commit derived outputs atomically for tick k .

3 Core Invariants (What Must Always Hold)

: Window coherence

For any analysis window W , all channels represent the same real-world time interval. Resampling or drift correction is applied consistently across channels.

: Stable framing

Frames have explicit sample rate, channel count, and a monotonic time anchor. Dropped frames are detected and recorded as gaps (never silently ignored).

: Geometry provenance

Array geometry and per-mic calibration parameters are versioned and included in the analysis configuration referenced by outputs.

: Reproducible ticks

Given identical PCM bytes and identical configuration, derived features and event outputs are identical (within a documented floating-point tolerance).

4 Algorithmic Spine (Advanced Ideas)

4.1 TDoA via GCC-PHAT (Why It Works)

Goal: estimate a time delay Δt between two microphone signals robustly under reverberation.

Concept model:

- compute cross-power spectrum $X_{12}(\omega)$
- apply PHAT weighting to emphasize phase alignment
- inverse FFT to obtain a correlation peak; $\arg \max$ gives Δt

Correctness depends on window coherence and sensible band limiting (avoid bands that produce spurious peaks).

4.2 Beamforming as Rewrite Semantics

Rather than “run a beamformer”, view beamforming as a deterministic rewrite:

- input: multi-channel frame + steering for angle θ
- rewrite: phase-align channels according to expected propagation delays
- output: an enhanced signal $y(t; \theta)$

This frames the correctness contract: geometry + timing + steering must agree.

4.3 Classification as a Gated Predicate

Classification is one gate among several quality/stability gates:

- SNR/quality metrics must exceed thresholds
- spatial estimate must be stable over a debounce window
- classifier probabilities must exceed class-specific thresholds

The output is not “a label”; it is a predicate with provenance.

5 Deployment Shapes (Edge vs Remote)

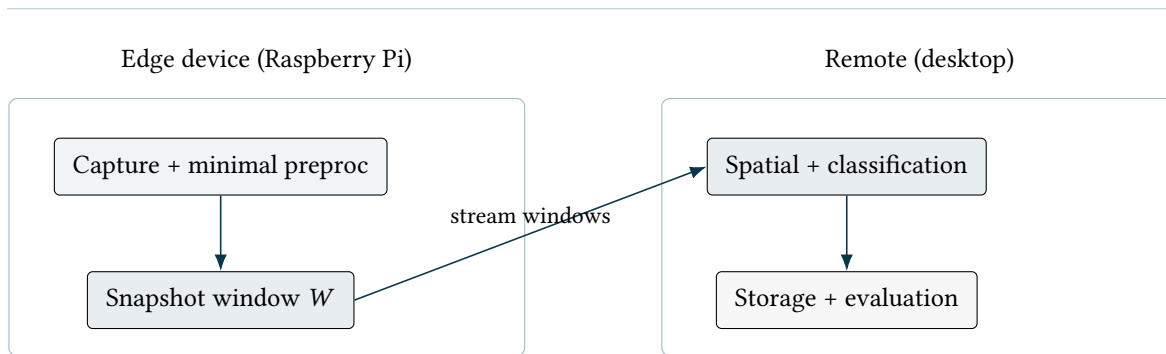


Figure 3. Two deployment shapes. Prefer placing the determinism anchor (window snapshot) as close to capture as possible.

6 Observability (What To Measure)

Treat metrics as first-class outputs:

- capture: frame drops, jitter, drift estimates
- preproc: per-channel RMS, noise floor, clipping rate
- spatial: GCC-PHAT peak sharpness, bearing stability, beamformer gain
- classifier: confidence/entropy trends, false trigger rate

7 Open Questions / Next Steps

- define a replay interchange format (multichannel WAV + JSON sidecar for geometry, framing, timestamps)
- choose and freeze canonical feature transforms (window, hop, mel bins, normalization) for determinism
- add an evaluation harness: known recordings + expected event traces